
Milestone-2 Report: Safety Verification of Chiron IR using Constrained Horn Clauses

Project Overview

Program verification provides mathematical guarantees that software behaves correctly, unlike testing, which can find bugs but never prove their absence. While the Chiron framework offers support for dynamic analysis techniques such as fuzzing, symbolic execution, and spectrum-based fault localization, it currently lacks the capability to *prove* that a program satisfies a given specification for *all* possible inputs.

This project bridges the gap between testing and verification in Chiron by implementing a PDR-based safety verification engine. We target properties natural to turtle graphics programs, such as spatial bounds on the turtle's position and invariants over program variables.

Team Information

Roll No.	Roll No.	Email ID
Aditi Khandelia	220061	aditikh22@iitk.ac.in
Arush Upadhyaya	220213	arushu22@iitk.ac.in
Kushagra Srivastava	220573	skushagra22@iitk.ac.in

Source Code

The complete implementation is hosted at: <https://github.com/chiron-verification/milestone-2>

- **Milestone-1 commit:** 0f99ade6d4aa2516f9e47e013ad06d29e8b8a2f7
- **Milestone-2 commit:** <TBD>

Core source files and responsibilities:

- **safety_properties.py:** Main verification API; parses user properties, applies checks/hints, and reports PASSED/FAILED/UNKNOWN with timing, and now refactored from an interactive CLI to callable `CHC_Verification(...)` with structured `ReturnValue`, timeout handling, property-scope control, and heading-grid pre-check/hint orchestration.
- **step_rules.py:** Encodes IR instructions into CHC transition rules used by the fixedpoint engine, with replaced trig bucket bounds with exact 15-degree trig encoding, introduced **BadHeading** rule generation, and added optimization-aware loop summarization and turn-safety shortcuts.
- **init_fixed_point.py:** Initializes solver state and base relations for default, specific, and universal modes, and now registers **BadHeading**, constrains universal initialization to heading-grid and integer user variables
- **z3_fixed_point.py:** Defines invariant/state-vector relations and mode-aware predicate structure and introduces the dedicated **BadHeading** relation in the same state signature as **Inv**.
- **variable_name_detection_in_IR.py:** Extracts variables from IR for building constraints and queries, and now introduces the shared `OptimizationLevel` enum and updates parser signatures/import path setup to align with the standalone `CHC-Verification` package layout.
- **heading_grid.py:** Encodes heading-grid checks to handle 15-degree alignment assumptions explicitly, and now added as a dedicated helper module exposing `heading_on_grid(h)` for reuse across initialization, transition encoding, and API-level assumptions.

- **optimization_helpers.py:** Provides static-analysis helpers for optimization decisions and loop simplification metadata, introduced to implement turn-safety dataflow, repeat-loop pattern detection, and summarizability checks used by `OptimizationLevel.BASIC`.

The correctness test suite (`correctness_tests/`) has substantially expanded across `test_default.py`, `test_specific.py`, and `test_universal.py`, along with many new `.tl` fixtures and shared helpers to improve coverage of arithmetic, control-flow, and heading-related properties.

In addition, a dedicated performance testing setup (`performance_tests/`) is added with mode-wise test runners, benchmark fixtures, and reusable harness/configuration files to measure solver behavior on deep nesting, wide-state, branch-heavy, and high-iteration workloads.

Current Progress

Overview of the progress made since the first milestone is as follows:

- **Pipeline refactoring into a callable API:** The verification flow is now exposed through `CHC_Verification(...)` with structured output fields (status, advice, timing, and per-property results) instead of an interactive-only CLI path.
- **Heading-grid safety checks integrated:** The verifier now models and queries heading-grid safety explicitly through a dedicated `BadHeading` relation before property checking.
- **Property scope feature added:** Properties can now be checked over all reachable states or restricted to terminating states only (`property_scope = "terminating"`).
- **Heading-grid hints integrated:** Users can either request verification of the heading-grid invariant (`check_heading_always_on_grid`) or force an assumption (`heading_on_grid_always`) for faster runs on hard cases.
- **Optimization pipeline added:** A `BASIC` optimization mode adds static IR analysis, turn-safety reasoning, and summarizable-loop compression before `CHC` querying, while `OptimizationLevel.NONE` retains the conservative baseline behavior.
- **Correctness tests:** An expanded test suite now covers a wider variety of programs and properties, including terminating-state properties, tests with heading-grid hints, and some tests with `OptimizationLevel.BASIC` to validate correctness under optimization.
- **Performance tests:** *TODO*

See Figure 1 for a visual overview of the main verification pipeline stages, responsibilities, and data flow, and the subsequent sections for detailed implementation notes on each major feature addition and refactor. The details of correctness and performance testing can be found in the Tests section.

Implementation Details: Callable Verification API

File: `CHC-Verification/safety_properties.py`

The prior interactive `__main__` flow was replaced by a callable entry point:

```
CHC_Verification(
    file_name,
    mode,
    user_properties,
    params=None,
    property_scope="all",
    hints=["check_heading_always_on_grid"],
    timeout_ms=60_000,
    optimization_level=OptimizationLevel.NONE,
)
```

This API returns a `ReturnValue` object carrying: `status`, `heading_grid_safe`, `build_time`, `solve_times`, `passing_properties`, `failing_properties`, and `unknown_properties`. The implementation first validates mode, scope, hint format, and specific-mode parameters, then builds the fixedpoint, applies assumptions/pre-checks, and evaluates properties in a controlled `eval` context with restricted builtins.

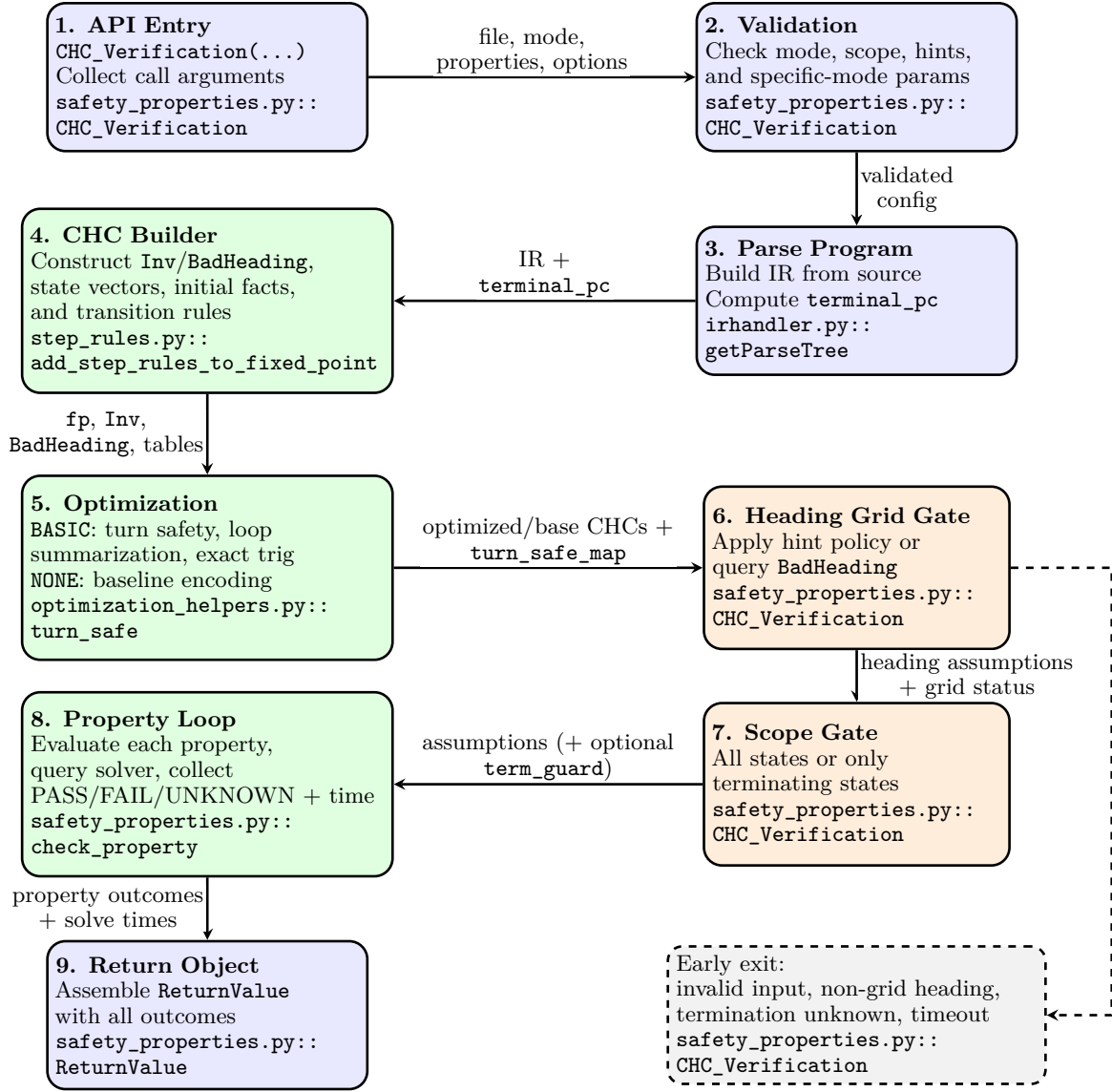


Figure 1: Main verification pipeline showing stage responsibilities and data artifacts flowing through parsing, CHC construction, heading/scope controls, and final result aggregation:

Implementation Details: Heading-grid Safety Check

Files:

1. *CHC-Verification/z3_fixed_point.py*
2. *CHC-Verification/init_fixed_point.py*
3. *CHC-Verification/step_rules.py*
4. *CHC-Verification/heading_grid.py*

The new pipeline introduces a dedicated relation `BadHeading(...)` at the invariant signature level and registers it in the fixedpoint context. Transition rules for turn commands can emit `BadHeading` consequences whenever `Not(heading_on_grid(next_heading))` is reachable, see Figure 2 for the workflow.

At API level, when checking is requested, the verifier queries `Exists(vars, BadHeading(state))`. The result semantics are: `sat -> heading_grid_safe = FAILED` (overall UNKNOWN), `unsat -> heading_grid_safe = PASSED` (continue), `unknown -> heading_grid_safe = UNKNOWN` (overall UNKNOWN).

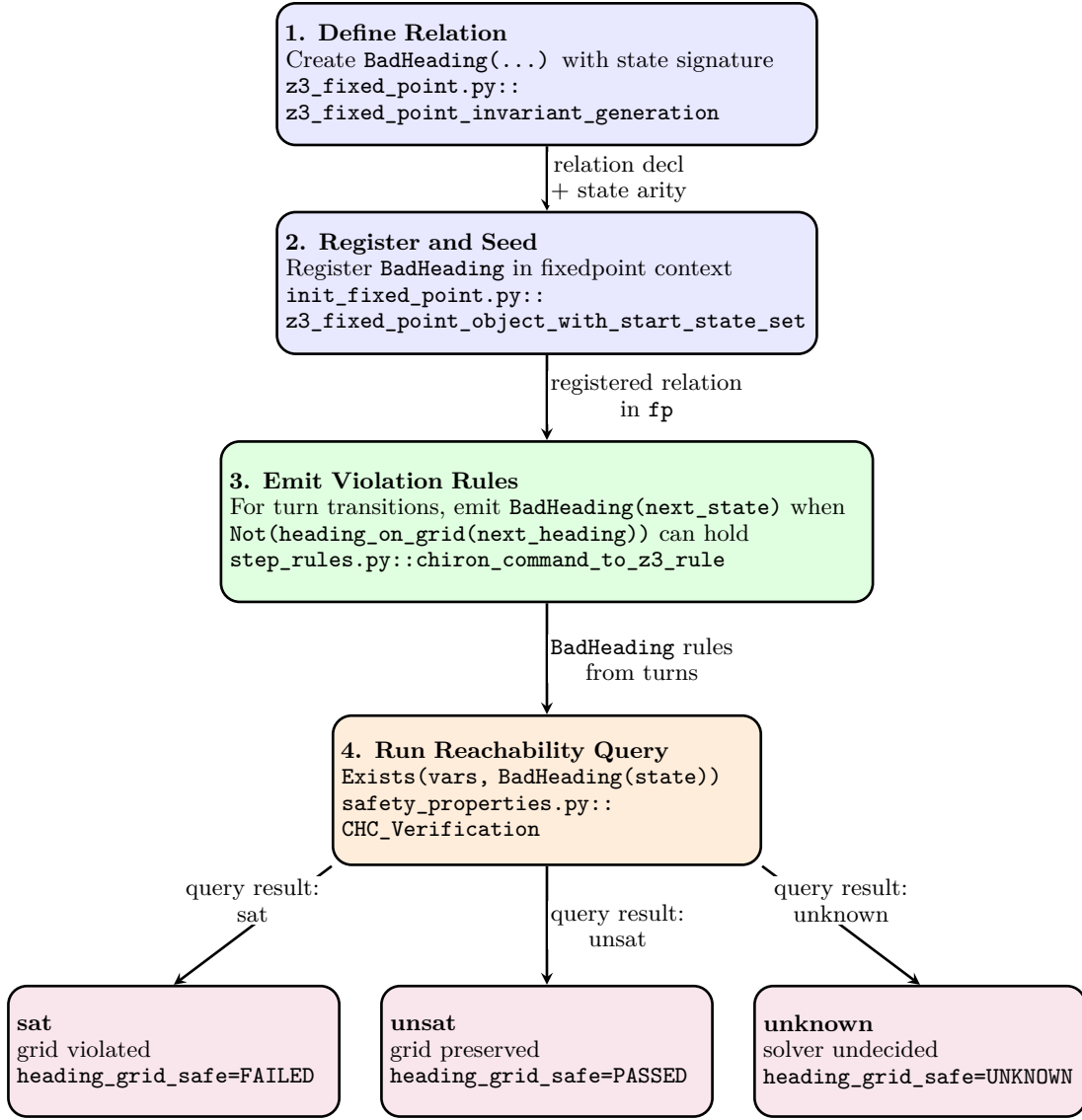


Figure 2: BadHeading workflow for proving whether heading remains on the 15-degree grid and mapping query outcomes to verification status:

This turns an implicit modeling assumption into an explicit proof obligation, improving interpretability of trig-sensitive verification outcomes.

Implementation Details: Property Scope

File: *CHC-Verification/safety_properties.py*

The new `property_scope` input supports "all" and "terminating". For terminating-only checks, the implementation:

1. computes terminal PC as `len(ir)`;
2. checks reachability of terminating states using `Exists(vars, Inv(state) & term_guard)`;
3. returns UNKNOWN early if termination is unreachable/unknown;
4. conjoins `term_guard` into assumptions for all property queries.

This keeps a single property-checking loop while changing the semantic domain of verification in a principled way.

Implementation Details: Optimization Pipeline

Files: *CHC-Verification/optimization_helpers.py*, *CHC-Verification/step_rules.py*

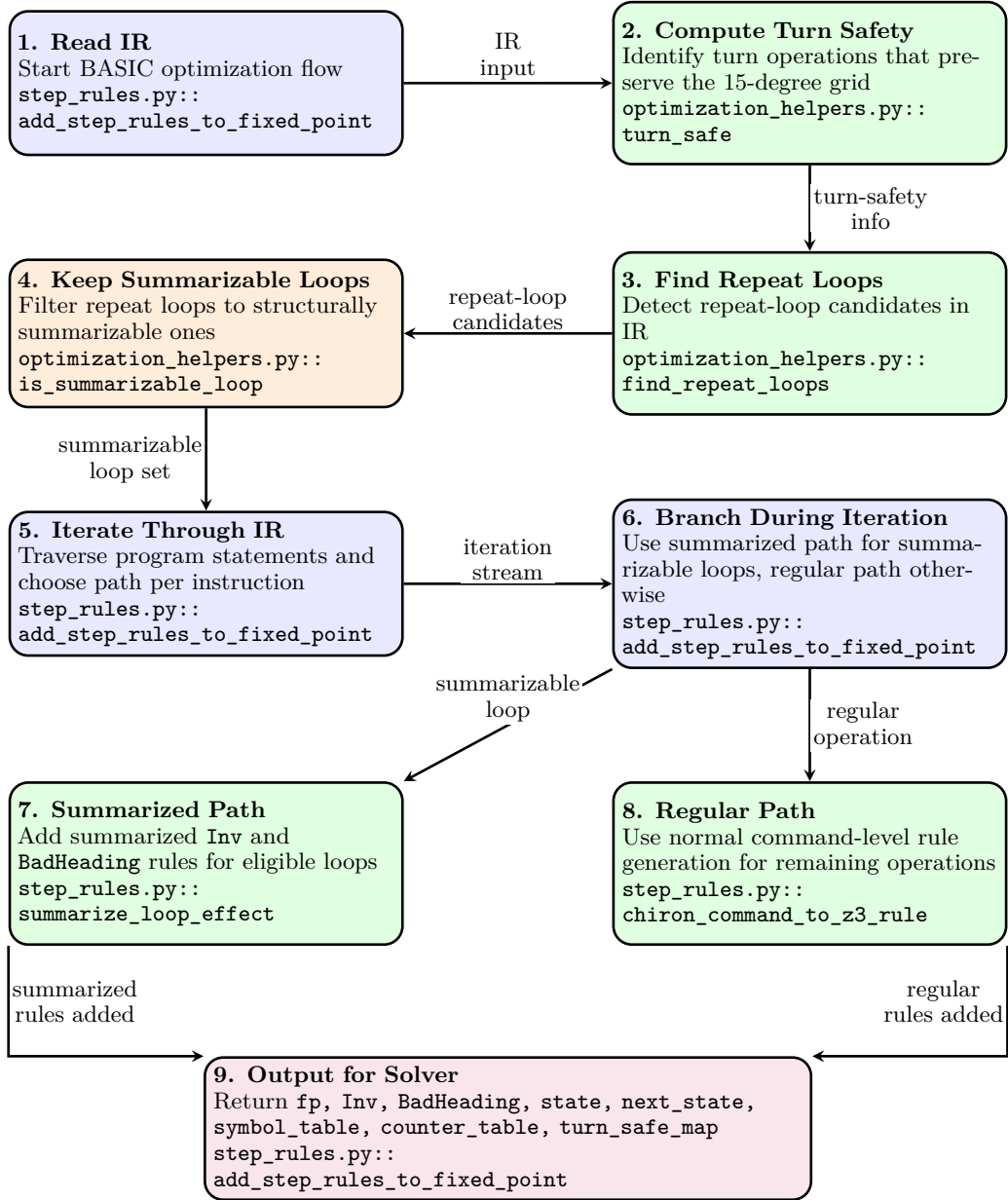


Figure 3: BASIC optimization workflow: read IR, compute turn safety, detect and filter repeat loops to summarizable loops, then during iteration emit summarized `Inv`/`BadHeading` rules for eligible loops and regular rules for other operations:

New optimization layer with two supported modes. In `OptimizationLevel.BASIC`, the verifier computes a static environment over IR control-flow to detect turn operations that provably preserve the 15-degree grid (`turn_safe`). It also detects canonical repeat-loop shapes through `find_repeat_loops(...)` and admits only structurally safe loops via `is_summarizable_loop(...)`.

For those loops, `step_rules.py` synthesizes a summarized transition from loop-entry PC directly to loop-exit PC and emits any required summarized `BadHeading` obligations. For non-summarized code, the original

instruction-wise rule generation remains active. This preserves soundness while reducing rule count and symbolic clutter on common deterministic repeat patterns.

An additional semantic tightening is that movement trig handling was upgraded from interval over-approximation buckets to exact 15-degree lookup values (`cos_sin_exact_z3`), aligning the transition encoding with the new heading-grid reasoning. The `OptimizationLevel.NONE` mode bypassed the optimization pipeline and performs standard verification. See Figure 3 for a visual overview of the optimization flow.

Implementation Details: Heading-grid Hints

File: CHC-Verification/safety_properties.py

Hints are parsed as a `StrEnum` with values `check_heading_always_on_grid` and `heading_on_grid_always`. The implementation validates hint type and value, then applies a precedence rule:

- if `heading_on_grid_always` is present, the solver skips the heading-grid query and directly injects `heading_on_grid(state[3])` into assumptions;
- otherwise, if `check_heading_always_on_grid` is present, the explicit `BadHeading` query is performed.

In `OptimizationLevel.BASIC`, if static analysis proves every turn is safe (`is_all_turn_safe`), the heading-grid check short-circuits to `PASSED`, avoiding unnecessary solver effort.

API Usage Guide

The expected calling pattern is:

```
from safety_properties import CHC_Verification
from variable_name_detection_in_IR import OptimizationLevel

class UserProperty:
    def __init__(self, name, expr):
        self.name = name
        self.expr = expr

props = [UserProperty("x_nonneg", "x >= 0")]

result = CHC_Verification(
    "correctness_tests/programs/loop_accum.tl",
    "default",
    props,
    property_scope="all",
    hints=["check_heading_always_on_grid"],
    timeout_ms=60_000,
    optimization_level=OptimizationLevel.BASIC,
)

print("Result:", result.status)
print("Heading Grid Safe:", result.heading_grid_safe)
print("Build Time:", result.build_time)
print("Solve Times:", result.solve_times)
```

Operational notes:

- Use `specific` mode with a parameter dictionary string (e.g., `"'x': 10"`);
- Use `property_scope="terminating"` only when termination-focused properties are intended;
- Prefer `check_heading_always_on_grid` unless the program is known to be heading-grid safe or the check is not relevant, as provides a good balance of soundness and performance.

Tests

Correctness Tests

The correctness suite resides in `CHC-Verification/correctness_tests/` and is executed with `pytest` over `unittest.TestCase`-based test classes.

Evaluation modules and responsibilities

File	Role in evaluation
test_default.py	Validates concrete-start-state verification in default mode, including arithmetic, geometric, pen-state, directional, heading-grid, trigonometric, and nested-loop behaviors.
test_specific.py	Validates parameterized initialization in specific mode, with boundary-sensitive outcomes under user-supplied parameter dictionaries.
test_universal.py	Validates symbolic/unconstrained initialization in universal mode, API-level validation errors, heading-grid handling, and terminating-scope properties.
helpers.py	Shared harness for loading programs, invoking CHC_Verification , building assertion contexts, and standardizing property checks.
README.md	Documents suite structure, mode coverage, fixture list, and execution commands.

Coverage snapshot

- Total collected correctness tests: **206**
- Mode split: **46** (default), **70** (specific), **90** (universal)
- Fixture programs in **programs/**: **63** **.tl** files
- Explicitly skipped tests: **6** timeout-marked heading/trig-heavy scenarios

Category coverage across modes

- **Default mode:** arithmetic invariants, geometric bounds, pen-state properties, directional/heading checks, and advanced nested control-flow fixtures.
- **Specific mode:** same semantic families under concrete parameter assignments, including multi-parameter and chained-update programs.
- **Universal mode:** unconstrained initial-state reasoning, relational arithmetic properties, API-validation behavior, and terminating-scope checks.

How to run

From repository root (recommended):

```
cd Chiron-Framework/ChironCore
source .venv/bin/activate
cd ../../CHC-Verification/correctness_tests
pytest -q
```

Useful variants:

```
# verbose mode
pytest -v

# collect-only sanity check
pytest --collect-only -q

# run one mode file
pytest test_default.py -q

# run one test class
pytest test_specific.py::TestSpecificArithmetic -q

# run one test case
pytest test_universal.py::TestUniversalAPI::test_invalid_mode_error -q
```

Fixture programs (.tl) used in correctness evaluation

Fixture file	Purpose in tests
--------------	------------------

assign_algebra.tl	Multi-variable algebraic assignment flow.
assign_basic.tl	Basic assignment and scalar update sanity checks.
branchy_non_grid.tl	Branch-induced heading behavior that can leave the 15-degree grid.
conditional.tl	Conditional branch semantics and path-sensitive invariants.
flower_nested_pen.tl	Nested loops with pen-state and geometric progression.
forward_square.tl	Square drawing using forward and 90-degree turns
goto_accum.tl	Coordinate accumulation with goto-based motion.
goto_computed.tl	Computed goto expressions with arithmetic state coupling.
loop_accum.tl	Loop-based accumulation and bound preservation.
loop_adv.tl	More complex loop interactions and value evolution.
loop_basic.tl	Basic counted-loop semantics and monotonic updates.
loop_cond.tl	Conditional logic embedded inside loops.
loop_nested.tl	Nested repeat control-flow behavior.
maze_counter.tl	Counter-heavy control-flow with parameterized transitions.
param_accumulate.tl	Parameterized accumulation behavior under concrete starts.
param_chain.tl	Chained assignments/updates from seeded parameters.
param_cond.tl	Parameter-driven branch selection behavior.
param_countdown.tl	Countdown loop with boundary-sensitive outcomes.
param_goto.tl	Parameterized goto endpoints and geometric constraints.
param_loop.tl	Parameterized loop evolution and final-state checks.
param_loop_move.tl	Looped movement with parameterized displacement.
param_nested_cond.tl	Nested conditionals keyed on concrete parameter ranges.
param_pen.tl	Parameter-gated pen-state transitions.
param_pen_cond.tl	Conditional pen behavior with threshold parameters.
param_scale.tl	Parameterized scaling and arithmetic relation checks.
pen_only.tl	Pen-up/pen-down semantics without movement confounders.
pen_toggle.tl	Alternating pen-state transitions across steps.
pen_with_var.tl	Variable-controlled pen transitions.
read_before_write.tl	Semantics when reads occur before first explicit assignment.
spiral_grid_goto.tl	Grid-respecting spiral-like goto traversal.
square_goto.tl	Square traversal using goto commands.
triangle_nested_pen.tl	Nested-loop triangle pattern with pen-state coupling.
turns_15.tl	Turn-only program preserving 15-degree heading lattice.
turns_only.tl	Pure heading updates without translational movement.
two_params.tl	Two-parameter relational arithmetic and order constraints.
u_abs_diff_branching.tl	Symbolic branch merges with absolute-difference invariants.
u_branch_always_mul_1	Branching turns that preserve multiple-of-15 headings.
u_branch_always_mul_1.tl	Advanced branch variant of heading-grid-preserving turns.
u_branch_merge_affine.tl	Affine arithmetic relations across merged symbolic paths.
u_branch_pen.tl	Symbolic branch effects on pen-state outcomes.
u_branch_pen_adv.tl	Advanced branch/pen interactions under symbolic starts.
u_clamp_to_zero.tl	Terminating-scope clamping and non-negativity guarantees.
u_euclid_avg.tl	Euclidean-style arithmetic identities over symbolic inputs.
u_fixed_left.tl	Deterministic left-turn heading evolution checks.
u_fixed_target.tl	Universal movement toward fixed geometric targets.
u_fixed_target_user.tl	Fixed-target relations tied to symbolic user variables.
u_heading_non_grid.tl	Demonstrates reachable non-grid headings (safety failure case).
u_init_core.tl	Baseline universal initial-state semantics checks.
u_loop_diagonal.tl	Symbolic diagonal loop movement and heading constraints.
u_loop_pen.tl	Loop-driven pen-state properties under symbolic starts.
u_mul_15_heading_o.tl	Multiplicative heading relations constrained to grid-safe turns.
u_net_heading_preserved.tl	Net heading-preservation under balanced turn sequences.
u_net_zero_turn.tl	Sequences with zero net heading change.
u_normalize_x.tl	Arithmetic normalization-style updates over symbolic values.
u_on_spot_turn.tl	In-place turning behavior and heading range invariants.
u_pen_no_touch.tl	Pen-state invariants where drawing state should remain unchanged.
u_pen_up.tl	Pen-up preservation guarantees across symbolic traces.
u_product_zero.tl	Product-based arithmetic relations and zero constraints.
u_same_postcondition.tl	Distinct symbolic paths converging to a shared postcondition.

<code>u_swap_with_temp.tl</code>	Swap semantics using a temporary variable.
<code>u_swap_without_tmp.tl</code>	Swap-style arithmetic relations without temporary storage.
<code>u_two_step_affine.tl</code>	Two-step affine transformations over symbolic inputs.
<code>u_xcor_fixed.tl</code>	Program preserving fixed x-coordinate relations.

Performance Tests

Analysis

Data and Visual Analysis

Strengths of the implementation

- **Broad verification coverage:** The pipeline supports default, universal and specific modes, now augmented with terminating-state properties (e.g., the turtle eventually reaches a target), enabling richer correctness questions.
- **Improved solver efficiency:** The new `OptimizationLevel.BASIC` path for universal properties reduces solver overhead and makes universal queries feasible on more complex programs, specifically those with loops
- **Heading-grid safety hints:** The introduction of heading-grid-based safety hints significantly reduces solver timeouts for geometric programs where the user is sure the turtle will only turn in multiples of 15 degrees.
- **Stronger empirical validation:** Performance tests and visual analyses help compare the impact of optimizations as heading-grid safety hints.
- **Improved test coverage:** The test suite now includes more diverse cases, covering both geometric and arithmetic properties as well as terminating state properties, and has been expanded to include stress tests for deeper loops and complex properties.

Limitations of the implementation

- **Limitation to heading-grid safety:** Movement commands that go out of the heading grid (e.g., 22.5 degree turns) lead to unknown verdict irrespective of the property.
- **Nonlinear arithmetic fragility:** Multiplicative constraints and non-linear relationships can still produce UNKNOWN results or long runtimes, especially when combined with geometric constraints.
- **Structured Geometric Reasoning:** Despite the hints and optimizations, some programs and properties involving complex geometric reasoning (e.g., intricate heading interactions) still lead to timeouts, indicating room for further encoding-level improvements.
- **Limited scalability validation:** While the test suite has grown, scalability for programs with extremely large loop counts or deeply nested structures remains unproven.

Future Plans

Milestone summary

- | | |
|-------------------------|--|
| Milestone 1 | Literature review and naive CHC pipeline - <i>completed</i> : CHC theory studied, Chiron-to-CHC semantics formalised, Z3 Fixedpoint playground implemented. First end-to-end pipeline with basic CHC encoding and property checking developed, supporting three modes: <code>default</code> , <code>universal</code> , <code>specific</code> , but with limited optimizations and test coverage. |
| Milestone 2 | End-to-end robust CHC verification pipeline - <i>completed</i> : This milestone focused on strengthening the full IR-to-CHC translation pipeline, adding support for checking property at program termination, restricting to 15-degree multiple grid for determinism in restricted case and support optimization for special cases. |
| Final Submission | Further optimization and user-friendly API - <i>pending</i> : The final milestone will focus on improving the pipeline's efficiency through further encoding optimizations and solver parameter tuning. Additionally, efforts will be directed towards developing a user-friendly API to enhance accessibility and usability for end-users. This phase will also include final documentation, a polished test suite, and a reproducible end-to-end tool. |

Planned work

(a) Optimization and robustness

The final milestone will prioritize:

- Reducing solver overhead by refining CHC encodings and addressing inefficiencies in trigonometric reasoning.
- Extending the benchmark suite with complex and computationally intensive programs.
- Minimizing UNKNOWN verdicts through encoding-level improvements and solver tuning.

(b) User-friendly API development

To make the pipeline more accessible, planned work includes:

- Designing and implementing a high-level API for easier integration with external tools.
- Providing concrete guarantee on what scenarios the user can expect to get a definitive answer and when they should expect an unknown.

(c) Final integration and deliverables

The final phase will consolidate all components into a cohesive tool, including:

- Stabilizing CLI outputs and ensuring reproducibility across environments.
- Completing the final report and preparing the project poster.
- Delivering a fully documented and user-friendly verification framework.

AI Use in the Project

In this project, we utilized AI tools to enhance our workflow and improve the quality of our work.

1. **Are you using AI for coding your implementation?** Yes, we used AI tools in this project.
2. **If yes, what models are you using?** We used models like OpenAI's GPT-5.2 Codex and GitHub Copilot.
3. **If using AI, how are you using AI and what percentage (approx) of your code is AI generated?** We used AI primarily for generating boilerplate code, suggesting improvements, and debugging. Approximately 40% – 50% of our code was generated or assisted by AI tools.
 - AI was primarily used for generating boilerplate code and usage of correct syntax, which was then reviewed and modified by us to fit our specific requirements.
 - We also used AI for testing, where it helped in generating test cases and suggesting improvements to our existing tests.
 - Additionally, AI was used for the generation of documentation and figures in the report, where it assisted in summarizing the directory structure and usage guide, as well as making the figures in the report with careful and structured instructions about the layout.

Declaration

- We declare that the work presented in this report is our own and has been carried out in accordance with the guidelines set out by the course.
- We have not copied any part of this report from any other source, and we have not submitted this report to any other course or institution for assessment.
- We understand that any violation of this declaration may result in disciplinary action.